视觉分析平台设计

整体设计

技术选型

- 模型训练
- 模型导出
- 平台开发

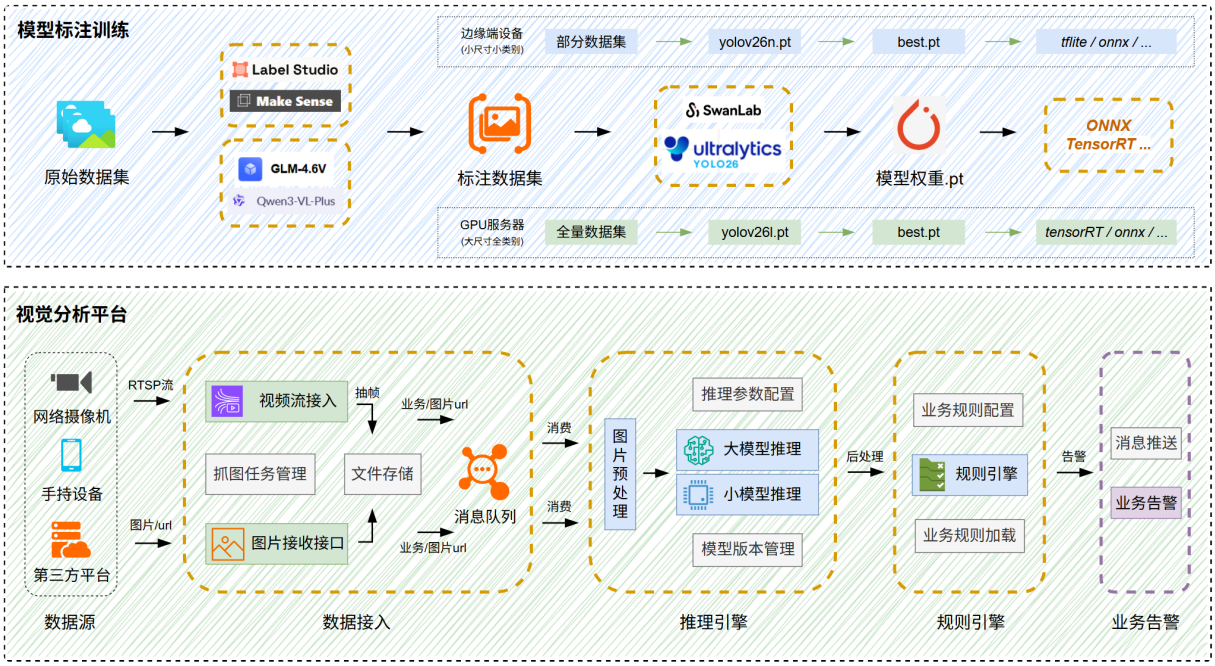
详细设计

- 抓图消息格式
- 推理结果格式
- 告警消息格式

算力评估

- 负载需求预估
- 模型推理吞吐
- GPU 算力评估
- CPU 算力评估
- 综合推荐配置

整体设计



技术选型

模型训练

模块	推荐技术	选型理由
数据标注	**Label Studio** / Make Sense	开源、支持多人协作、多格式导出，可半自动标注结合大模型（GLM-4.6V、Qwen3-VL）
训练框架	**Ultralytics YOLOv26**	边缘优先设计，NMS-Free、无 DFL、CPU 速度提升 43%，完美适配边缘设备
预训练模型	YOLOv26n / YOLOv26s / YOLOv26l / YOLOv26x	5 种尺寸可选，nano 仅 2.4M 参数，适合极端边缘场景
模型版本管理	Git+ 对象存储 MinIO	追踪数据集、模型权重版本，便于回滚与协作
训练分析平台	SwanLab	国产开源，完美适配 PyTorch、Ultralytics， 训练实时监控、超参数

YOLOv26 模型版本选择建议:

部署目标	推荐版本	参数量	FLOPs	适用场景
极端边缘（树莓派、RK3566）	YOLOv26n	2.4M	5.4B	实时性要求极高，检测目标 ≤3 类
边缘主流（Jetson Nano、RK3588）	YOLOv26s	9.5M	20.7B	精度与速度均衡，检测目标 5-10 类
GPU 服务器（中精度）	YOLOv26m / l	20.4M / 24.8M	68.2B / 86.4B	全量类别检测，精度优先
GPU 服务器（高精度）	YOLOv26x	55.7M	193.9B	极致精度，算力充足

模型导出

部署目标	推荐格式	转换工具	YOLOv26 特别说明

GPU服务器（NVIDIA）	TensorRT `.engine`	`yolo export format=engine`	YOLOv26 移除 DFL/NMS 后，TensorRT 导出更简洁，延迟更低
边缘端（NVIDIA Jetson）	TensorRT `.engine`	`yolo export format=engine`	统一格式，利用 Jetson 硬件加速
边缘端（ARM/树莓派/RK）	ExecuTorch `.pte`	`yolo export format=executorch`	YOLOv26 原生支持，XNNPACK 后端，CPU 推理比 PyTorch 快 2 倍以上
边缘端（Intel CPU/NUC）	OpenVINO `.xml`	`yolo export format=openvino`	Intel 平台专用优化
移动端（iOS/Android）	ExecuTorch `.pte`	`yolo export format=executorch`	原生支持 iOS/Android，与 PyTorch 生态无缝集成
跨平台通用	ONNX `.onnx`	`yolo export format=onnx`	中间格式，便于调试和转换

平台开发

组件	推荐技术	版本	选型理由
编程语言	Python	3.8+	生态丰富，与 PyTorch、YOLO 深度绑定，3.11 性能更优，兼容性好
Web 框架	FastAPI	0.115+	异步、高性能、自动生成 OpenAPI 文档，原生支持 Pydantic 校验
ASGI 服务器	Uvicorn	0.30+	轻量、快速，支持多 worker 进程
模型推理引擎	Ultralytics YOLO	8.4.0+	统一 Python API，原生支持 YOLOv26 和 ExecuTorch 导出
消息队列	Apache Kafka	3.6+	高吞吐、持久化、支持多消费者组，适合削峰填谷、解耦数据生产与消费

任务存储	PostgreSQL	12+	存储任务配置，支持 JSON 字段存储灵活配置
拉流/抽帧	FFmpeg	6.0+	业界标准，支持 RTSP/RTMP/HTTP-FLV 等协议，可集成 Python 子进程调用
图片存储	MinIO	-	对象存储，统一 URL 访问，便于推理服务获取图片
规则引擎	自研 / rule-engine	-	简单场景可用 Python 条件判断 + 配置化；若规则复杂，可选开源工具；
容器化	Docker	19+	快速发布部署

详细设计

抓图消息格式

Topic设计

- 使用单一Topic，通过消息的 `scene` 字段区分业务场景
- 按 `scene` 字段哈希分区

消息格式

```
{
  "message_id": "uuid",           // 消息id
  "timestamp": 1711382400000,     // 消息产生时间
  "scene": "gas",                 // 业务场景
  "stream_id": "cam_001",         // 视频流标识
  "capture_time": 1711382400000,  // 图片抓取时间
  "image_url": "https://xxx/images/2024-03-25/cam_001_1711382400000.jpg",  // 图片访问URL
  "image_width": 1920,            // 图片尺寸，便于预处理
  "image_height": 1080,
  "extra": {                      // 扩展字段
    "camera_location": "xx区xx路",
    "trigger_type": "schedule"  // 或 "event", "api"
  }
}
```

推理结果格式

所有模型推理结果结构统一，便于后续规则引擎统一处理。

```
{
  "message_id": "uuid",           // 与输入消息关联
  "timestamp": 1711382401000,     // 推理完成时间
  "scene": "gas",                 // 业务场景
  "stream_id": "cam_001",         // 视频流标识
  "capture_time": 1711382400000,  // 图片抓取时间
  "inference_time_ms": 45,        // 本次推理耗时
  "model_name": "yolov8l_gas_v2", // 使用的模型标识
  "objects": [                    // 检测到的目标列表
    {
      "class_id": 0,                // 类别ID，训练定义的类别索引
      "class_name": "valve",        // 类别名称（便于规则引擎可读）
      "confidence": 0.92,           // 置信度
      "bbox": [100, 200, 300, 400], // [x1, y1, x2, y2] 像素坐标
      "attributes": {               // 可选：目标属性，如颜色、方向等
        "status": "open"
      }
    },
    {
      "class_id": 1,
      "class_name": "leak",
      "confidence": 0.78,
      "bbox": [500, 600, 550, 630]
    }
  ],
  "image_meta": {                   // 可选：图片元数据
    "url": "https://oss...",
    "width": 1920,
    "height": 1080
  },
  "extra": {                        // 扩展信息
    "camera_location": "xx区xx路"
  }
}
```

告警消息格式

```json
{
  "alert_id": "uuid",              // 唯一id
  "timestamp": 1711382402000,      // 告警产生时间
  "alert_level": "critical",       // critical, warning, info
  "alert_type": "gas_leak",        // 告警类型（如 leak, intrusion, occupancy）
  "scene": "gas",                  // 业务场景
  "stream_id": "cam_001",          // 视频流标识
  "capture_time": 1711382400000,   // 图片抓取时间
  "alert_time": 1711382402000,     // 告警规则触发时间
  "trigger_condition": "leak confidence > 0.7 and duration > 3 frames",  // 触发条件的描述
  "related_objects": [             // 触发告警的目标列表（从推理结果中选取）
    {
      "class_name": "leak",
      "confidence": 0.78,
      "bbox": [500, 600, 550, 630],
      "attributes": {}
    }
  ],
  "evidence": {                    // 证据图片/视频片段URL，供业务应用直接展示
    "image_url": "https://xxx/images/2024-03-25/cam_001_1711382400000.jpg",
    "video_clip_url": "https://xxx/clips/2024-03-25/cam_001_1711382400.mp4", // 可选
    "snapshots": ["url1", "url2"]  // 可选，多帧证据
  },
  "location": {                    // 位置信息，便于GIS展示或工单派发
    "camera_id": "cam_001",
    "camera_name": "燃气站东门",
    "address": "xx区xx路100号",
    "longitude": 116.397,
    "latitude": 39.907
  },
  "extra": {
    "rule_id": "gas_leak_rule_v2"  // 扩展信息
  }
}
```

算力评估

负载需求预估

以下数据按照产品收集的数据初步推算：

- **流式推理**：45 路 × 5 帧/秒 = 225 次推理/秒
- **图片推理**：约 9 次推理/秒（包含所有推图场景的总和）
- **总吞吐量需求**：234 次推理/秒

流式推理对延迟敏感（单帧需 < 50ms），必须使用 GPU 加速；图片推理延迟容忍度高，可由 CPU 或 GPU 空闲算力处理。

模型推理吞吐

NVIDIA T4 (16GB) TensorRT FP16，批大小 = 1

模型	单张推理延迟	单卡理论吞吐	满足 234 次/秒所需卡数
YOLOv26n	4 ms	250 次/秒	1（接近上限）
YOLOv26s	6 ms	166 次/秒	2
YOLOv26m	12 ms	83 次/秒	3
YOLOv26l	18 ms	55 次/秒	5
YOLOv26x	28 ms	35 次/秒	7

实际生产环境需预留 20-30% 余量，且多模型切换、预处理等会额外消耗算力。

GPU 算力评估

场景	推荐 GPU	说明
轻量模型为主（YOLOv26n/s）	1× RTX 4090 或 1× A6000	单卡吞吐充裕（330+ QPS），可覆盖流式+图片推理
混合模型（含 m/l 或同时加载多场景模型）	2× RTX 4090 或 1× A100 40G	保证余量，支持多场景模型并行加载

说明：

- RTX 4090：消费级旗舰，24G显存，算力强、性价比高，适合对成本敏感的推理场景。
- A6000：企业级 48GB 显存，支持 ECC 内存，适合 7×24 小时生产环境。

CPU 算力评估

仅图片推理

以 Intel Xeon Gold 5318Y @ 2.1GHz，ONNX Runtime 为例：

模型	单张延迟	单核吞吐	满足 9 次/秒所需核心数
YOLOv26n	150 ms	6.7 次/秒	2 核（余量充足）
YOLOv26s	220 ms	4.5 次/秒	3 核

YOLOv26m	350 ms	2.9 次/秒	4 核

图片推理负载极低，建议直接复用 GPU 服务器空闲 CPU，或使用 2 核轻量虚拟机独立部署。

若全部使用 CPU 处理流式推理：

- 单核吞吐仅 4-7 次/秒，需 30-50 核才能满足 225 QPS，成本远高于 GPU，且无法保证实时性（延迟 150ms+）。

综合推荐配置

方案	GPU	CPU	内存	说明
高性价比方案	1× RTX 4090	2 核	16 GB	单卡满足流式+图片，预留余量
企业级方案	1× A6000	2 核	32 GB	48GB 显存支持多场景模型同时加载

图片推理部分（9 QPS）可由 GPU 空闲算力或 2 核 CPU 轻松处理，无需额外资源。

补充说明：

- 若未来流式路数增加，可水平扩展 GPU 节点（如增加至 2× RTX 4090）。
- 若所有场景均为纯图片推理（无流式），则单台 2 核 4GB 虚拟机即可稳定支持至少 **3000 路摄像头每 5 分钟一次的推理任务。**